

Pipelining e Vetorização

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long int size = 99999999 * 8;

double *init() {
    double *vec = (double *)malloc(sizeof(double) * size);

    for (size_t i = 0; i < size; i++) {
        vec[i] = (i + 5) / 500.;
    }

    return vec;
}

double somatorio_base(double *vec) {
    double soma = 0;

    for (int i = 0; i < size; i++) {
        soma += vec[i];
    }

    return soma;
}

double somatorio_duplo(double *vec) {
    double soma1 = 0;
    double soma2 = 0;

    for (int i = 0; i < size; i += 2) {
        // Aqui inicia pipelining de mais de uma operação
        soma1 += vec[i];
        soma2 += vec[i + 1];
    }

    return soma1 + soma2;
}

double somatorio_quadruplo(double *vec) {
    double soma1 = 0;
    double soma2 = 0;
    double soma3 = 0;
    double soma4 = 0;

    for (int i = 0; i < size; i += 4) {
        // Aqui usando o meu notebook não fez diferença, ficou igual ao somatório
        // duplo
        soma1 += vec[i];
        soma2 += vec[i + 1];
        soma3 += vec[i + 2];
        soma4 += vec[i + 3];
    }
}
```

```

    return soma1 + soma2 + soma3 + soma4;
}

int main() {
    double *vec = init();

    struct timespec start, end;
    double elapsed;

    clock_gettime(CLOCK_MONOTONIC, &start);
    double soma1 = somatorio_base(vec);
    clock_gettime(CLOCK_MONOTONIC, &end);

    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("soma 1: %f\n", soma1);
    printf("Elapsed wall time: %f seconds\n", elapsed);

    clock_gettime(CLOCK_MONOTONIC, &start);
    double soma2 = somatorio_duplo(vec);
    clock_gettime(CLOCK_MONOTONIC, &end);

    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("soma 2: %f\n", soma2);
    printf("Elapsed wall time: %f seconds\n", elapsed);

    clock_gettime(CLOCK_MONOTONIC, &start);
    double soma3 = somatorio_duplo(vec);
    clock_gettime(CLOCK_MONOTONIC, &end);

    elapsed = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("soma 3: %f\n", soma3);
    printf("Elapsed wall time: %f seconds\n", elapsed);

    return 0;
}

// ❷ ~ /programming/hpc ~ took 7s R ~ at 18:00:49 ))
// > clang task2.c -O0
//
// ❸ ~ /programming/hpc ~ at 18:00:57 ))
// > ./a.out
// soma 1: 639999994400000.875000
// Elapsed wall time: 1.829295 seconds
// soma 2: 639999994400000.000000
// Elapsed wall time: 0.961911 seconds
// soma 3: 639999994400000.000000
// Elapsed wall time: 0.962176 seconds
//
// ❹ ~ /programming/hpc ~ took 7s R ~ at 18:01:05 ))
// > clang task2.c -O2
//

```

```
// ❶ ~ /programming/hpc at 18:01:08 ))
// > ./a.out
// soma 1: 639999994400000.875000
// Elapsed wall time: 0.841715 seconds
// soma 2: 639999994400000.000000
// Elapsed wall time: 0.447749 seconds
// soma 3: 639999994400000.000000
// Elapsed wall time: 0.446850 seconds
//
// ❷ ~ /programming/hpc took 4s R at 18:01:13 ))
// > clang task2.c -O3
//
// ❸ ~ /programming/hpc at 18:02:23 ))
// > ./a.out
// soma 1: 639999994400000.875000
// Elapsed wall time: 0.865701 seconds
// soma 2: 639999994400000.000000
// Elapsed wall time: 0.492016 seconds
// soma 3: 639999994400000.000000
// Elapsed wall time: 0.489958 seconds
```